JavaScript Competence: General Description

One description covering all levels. Levels are cumulative: a higher level includes everything from the lower levels.

1. Purpose and scope

This document describes what it means to be competent in JavaScript. It is used as reference material for creating and reviewing test questions. It does not split the competence into separate documents per level. Instead, each topic below says what is expected at Level 1 and what is added at Level 2. Questions must stay within the topics and depth described here.

2. What JavaScript competence means

A competent JavaScript developer can write, read, debug and maintain programs in modern JavaScript (ES2015 and newer), work with the browser and with Node.js, use the standard library and common tooling, and communicate with web APIs. Competence is shown by correctly applying concepts to short, realistic tasks, not only by recalling definitions. This document covers plain JavaScript. Frameworks and TypeScript are out of scope (see section 5).

3. Competence areas by level

Area	Level 1: foundation	Level 2: adds
Language fundamentals	Primitive types and objects, `let` and `const` (and why to avoid `var`), operators, conditionals, loops, template literals, comparing with `===`, truthy and falsy values, `typeof`.	Type coercion and the surprises of `==`, `null` versus `undefined`, optional chaining (`?.`) and nullish coalescing (`??`), destructuring, spread and rest, predicting the output of short code.
Functions and scope	Declaring functions, parameters and return values, arrow functions, default parameters, block scope versus function scope.	Closures, hoisting and the temporal dead zone, higher-order functions and callbacks, how `this` behaves in different call styles, arrow functions versus regular functions.
Objects and arrays	Creating objects and arrays, reading and changing properties, looping over arrays, `push`, `pop`, `slice`, `includes`, `JSON.stringify` and `JSON.parse`.	Array methods `map`, `filter`, `reduce`, `find`, `sort`; copying versus referencing, shallow versus deep copy, `Map` and `Set`, prototypes and classes at a basic level, immutability habits.
Asynchronous code	What asynchronous means, `setTimeout`, the idea of a callback, basic use of promises (`.then` and `.catch`).	`async` and `await`, running promises in parallel with `Promise.all`, the event loop and the order in which code runs, handling errors in async code, avoiding callback nesting.
Browser and DOM	Selecting elements with `querySelector`, changing text and attributes, handling events such as `click`, reading form input, using the browser console.	Creating and removing elements, event delegation and bubbling, `localStorage`, when scripts run, avoiding unnecessary DOM updates, basic security habits such as not inserting untrusted text as HTML.
Web requests and APIs	What HTTP is, requests and responses, JSON, making a request with `fetch`, reading the response.	Request methods and headers, status codes, checking `response.ok`, sending JSON, same-origin policy and CORS at a basic level, handling loading and error states.
Error handling	What an error is, `try`/`catch`, reading an error message and stack trace in the console, the difference between a syntax error and a runtime error.	Throwing errors, custom error messages, `finally`, handling rejected promises, defending against missing or wrongly typed data.

Area	Level 1: foundation	Level 2: adds
Modules and tooling	Using `npm` to install packages, running a script, Git basics (clone, commit, push, pull, branch), debugging with browser developer tools.	ES modules (`import` and `export`), `package.json` and dependencies, Node.js basics, a linter and formatter, writing and running simple unit tests, Git workflows with pull requests.
Code quality	Meaningful names, readable formatting, small functions, comments only where useful.	Avoiding duplication, pure functions and side effects, simple refactoring, reading and reviewing someone else's code, reasoning about edge cases.

4. How the levels differ

- **Level 1** checks that the person knows the basic building blocks and can use them in short, single-concept tasks. Typical questions: name the right keyword or method, identify what a line of code does, pick the correct basic structure.
- **Level 2** checks that the person can combine several concepts, read slightly longer code, choose between options and explain why. Typical questions: predict the result of a code fragment, explain why a closure or async example behaves as it does, spot a bug or a better approach.
- A question belongs to a level by the depth of reasoning it needs, not by the topic alone. The same topic can appear at both levels with different depth.

5. Out of scope

The following are not part of this competence unless a request explicitly includes them:

- TypeScript and other languages that compile to JavaScript.
- Frontend frameworks and libraries (React, Vue, Angular, Svelte, jQuery).
- Server frameworks and back-end design (Express, databases, authentication).
- HTML and CSS beyond what is needed to work with the DOM.
- Engine internals and performance tuning (garbage collection, JIT, bundler configuration).
- Generators, proxies, web workers, WebAssembly and other advanced features.
- Legacy patterns from before ES2015, and proposals that are not yet part of the standard.

6. Guidance for writing questions

- Each question tests one clear idea and has one unambiguously correct answer.
- Use only topics listed above, at the depth of the requested level.
- Prefer short code examples and realistic situations over pure definitions.
- Wrong answer options should be plausible and based on common mistakes, not on trick wording.
- Avoid questions that depend on obscure quirks of the language or on a specific browser version.
- If a question needs knowledge from the out-of-scope list, it should not be generated.